

A Pinch of Python

Python Language Lab Manual

Dr. Shibaji Banerjee and Dr. Suparna Roychowdhury

Department of Physics

St. Xavier's College (Autonomous), Kolkata

B.Sc. Physics Semester I (NEP)

July – December 2026

A gentle introduction to Python through turtle graphics, physics calculations, and the joy of discovery.

Copyleft © 2026 — Department of Physics, St. Xavier's College, Kolkata

Welcome to A Pinch of Python

Fact 1: About This Manual
This manual is designed for B.Sc. Physics Semester I students at St. Xavier's College (Autonomous), Kolkata. We believe learning programming should be joyful, visual, and connected to real physics. Through simple interactive sessions, beautiful turtle drawings, DNA string manipulation, and gravitational force calculations, you will master Python fundamentals while having fun.

Fact 2: How to Use This Manual

- Start the Python shell by typing `python` in terminal
- Try every example at the `>>>` prompt
- Complete mandatory assignments first
- Move to advanced ones when ready
- Use `help()` freely — it's your best friend!

Getting Started

Fact 3: Interactive Session — Type `python` Open terminal → type `python` → see `>>>`

```
>>> print("Hello St. Xavier's!")
Hello St. Xavier's!
```

Fact 4: Python is Case-Sensitive `Name` ≠ `name`

Fact: See What's in Your Python Workspace — Only 3 Commands

```
>>> dir()          # Works everywhere. Shows all names in current workspace
                  # (variables, functions, imported stuff)

>>> %who           # Only in Jupyter/IPython. Simple list of your variables

>>> %whos          # Only in Jupyter/IPython. Detailed: name, type, value/info
```

After your classic turtle session:

```
>>> from turtle import *
>>> reset()
>>> test = 4
>>> dir()              # you'll see 'test', 'forward', 'right',
                       'reset', etc.
>>> %whos              # (in Jupyter) shows: Variable   Type
                        Data/Info
```

Note: Only dir() works in normal Python shell — always use it!

Fact: Comments — Single, Double, Triple Quotes & Apostrophe Handling Python supports three ways to write comments:

- **Single-line comment:** `# This is a comment`
- **Multi-line string (used as comment):**
  """
  This is a
  multi-line comment
  """
- **Also works with triple single-quotes:** `'''like this'''`

Important: Triple-quoted strings are ignored if not assigned to a variable — perfect for long comments or docstrings.

Problem with apostrophe in "St. Xavier's": If you use single quotes: `'Hello St. Xavier's'` → **SyntaxError!** Python thinks the string ends at `St.`.
Solutions (all of them work):

```
# 1. Use double quotes (easiest)
print("Hello St. Xavier's")

# 2. Escape the apostrophe with backslash
print('Hello St. Xavier\'s')

# 3. Use triple quotes (great for long text)
print("""Hello St. Xavier's College""")

# 4. Triple single-quotes also work
print('''Hello St. Xavier's College''')
```

Best practice: Always prefer double quotes for strings containing apostrophes — clean and readable!

Fact 5: Check Version

```
>>> import sys
>>> print(sys.version)
```

The sys module
The `sys` module provides access to system-specific parameters and functions. Use `import sys` to load it. Common uses: `sys.version` → shows Python version, `sys.argv` → command-line arguments, `sys.path` → module search paths, `sys.exit()` → exit the program.

Fact: sys.argv — Command-Line Arguments in Python
When you run a Python script from terminal, you can pass extra values (arguments). `sys.argv` is a list that contains: - `sys.argv[0]` → name of your script - `sys.argv[1]`, `[2]`, ... → the values you typed after the script name

```
# Save as gravity.py
import sys
print(sys.argv)                    # shows everything typed

if len(sys.argv) != 4:
    print("Usage: python gravity.py <mass1> <mass2> <distance>")
else:
    m1 = float(sys.argv[1])
    m2 = float(sys.argv[2])
    r  = float(sys.argv[3])
    G  = 6.6743e-11
    F  = G * m1 * m2 / r**2
    print(f"Force = {F:.2e} N")
```

Run from terminal (not inside Python shell!):

```
$ python gravity.py 5.972e24 7.342e22 3.844e8
['gravity.py', '5.972e24', '7.342e22', '3.844e8']
Force = 1.98e+20 N
```

Another quick example — draw square with custom size:

```
# sqr_cmd.py
from turtle import *
import sys
size = 100
if len(sys.argv)<2:
    print('Arguements need to be 2 exactly')
else:
    float(sys.argv[1])

for i in range(4):
    forward(size); right(90)
done()
```

Run: `python sqr_cmd.py 200` → draws a 200-unit square
Key points for Physics lab:

- Always check `len(sys.argv)` first
- Convert arguments using `float()` or `int()`
- Use `sys.argv` when you want to run the same script with different data

Now your scripts behave like real scientific tools!

Fact: Clear the Terminal Screen Using os Module
To clear the Python interpreter screen (like `clear` in Linux or `cls` in Windows):

```
import os

# Works on Windows
os.system('cls')

# Works on Linux / macOS
os.system('clear')

# One-liner that works everywhere
os.system('cls' if os.name == 'nt' else 'clear')
```

Perfect for your lab — add this at the top of any script where you are using the `turtle` module:

```
import os
os.system('cls' if os.name == 'nt' else 'clear')
print("Welcome to Physics Python Lab!")
from turtle import *
reset()
```

Why this works:

- `os.name` is `'nt'` on Windows, `'posix'` on Linux/macOS
- `os.system()` runs shell commands
- `cls` = Windows clear, `clear` = Unix clear

Now you can start every session with a **clean, beautiful screen** — just like a fresh blackboard!
(Pro tip: Add the one-liner to all your .py files — instant professional look!)

Fact: Variables and Variable Casting

Variables are containers for storing data values. Python has **no command** for declaring a variable — it is created the moment you assign a value.

```
>>> name = "Ram"          # string
>>> age = 21              # integer
>>> height = 5.9          # float
>>> is_student = True     # boolean
```

Rules for variable names:

- Must start with a letter or underscore _
- Can contain letters, numbers, underscores
- Case-sensitive: `Age` ≠ `age`
- Avoid Python keywords (like `class`, `for`)

Variable Casting = Explicitly converting a variable from one type to another.

```
>>> x = "25"              # string
>>> int(x)                #  25 (integer)
>>> float(x)              # 25.0
>>> x + "5"               # "255" (concatenation)

>>> y = 10
>>> str(y) + " meters"    # "10 meters"

>>> z = "3.14"
>>> float(z) * 2          # 6.28

>>> age = "21"
>>> int(age) + 1          # 22 (now usable in math!)
```

Physics Example:

```
>>> mass_str = input("Enter mass (kg): ")   # user types "5.972e24"
>>> mass = float(mass_str)                  # now safe to use in F
    = G*m1*m2/r^2
```

Common Error:

```
>>> "25" + 5              # TypeError! Cannot mix string and int
```

Fix it with casting:

```
>>> "25" + str(5)         # "255"
>>> int("25") + 5         # 30
```

Best Practice: Always cast input or file data before calculations!

Built-in Data Types – Complete Table

Data Type	Example	Description
int	`x = 42`	Whole numbers
float	`G = 6.6743e-11`	Floating-point with scientific notation
complex	`z = 2+3j`	Complex numbers
str	`dna = "gcatgact"`	Immutable text (single/double/triple quotes)
bool	`flag = True`	True or False
list	`L = [1, 2, 3]`	Mutable ordered sequence
tuple	`t = (1, 2, 3)`	Immutable ordered sequence
dict	`d = {"Ram":21, "Riya":23}`	Key-value pairs
set	`s = {1, 2, 3}`	Unordered unique elements
None	`x = None`	Null value

Fact 6: Examples of Each

```
x = 42
G = 6.6743e-11
z = 2+3j
dna = "gcatgact"
flag = True
L = [1, 2, 3]
t = (1, 2, 3)
d = {"A":10, "C":5}
s = {1, 2, 3}
nothing = None
```

Fact 7: Strings are Immutable

```
s = "hello"
# s[0] = 'H'   # TypeError!
```

Fact 8: String Operations
Strings are **immutable** — once created, they cannot be changed. Python provides powerful built-in methods for manipulation:

```
dna = "  gCaTgAcTaTtCcTtGaC  "

# 1. Case conversion
print(dna.lower())      # "  gcatgactattccttgac  "
print(dna.upper())      # "  GCATGACTATTCCTTGAC  "

# 2. Remove whitespace
clean = dna.strip()     # "gCaTgAcTaTtCcTtGaC"
print(clean)

# 3. Count occurrences
print(clean.count('a'))   # 4
print(clean.count('A'))   # 1
print(clean.count('act')) # 1

# 4. Find position
print(clean.find('act'))  # 5 (first occurrence)
print(clean.find('xyz'))  # -1 (not found)

# 5. Replace
upper_a = clean.replace('a', 'A')
print(upper_a)

# 6. Split into list
parts = clean.split('t')
print(parts)  # ['gCaTgAcTaT', 'TcCtTGaC']

# 7. Join list back
joined = "---".join(parts)
print(joined)

# 8. Check start/end
print(clean.startswith('gC'))   # True
print(clean.endswith('GaC'))    # True

# 9. Slicing (like list)
print(clean[2:8])       # "aTgAcT"
print(clean[-6:])       # "TtGaC" (last 6 chars)
print(clean[::-1])      # reverse string
```

Apostrophe in "St. Xavier's" — Fixed!

```
# Wrong: SyntaxError!
# print('Hello St. Xavier's')

# Correct ways:
print("Hello St. Xavier's")           # double quotes
print('Hello St. Xavier\'s')          # escape with backslash
print("""Hello St. Xavier's College""")  # triple quotes
```

Physics Example — Format Numbers

```
G = 6.6743e-11
print(f"G = {G:.2e}")         # G = 6.67e-11
print("G = {:.2e}".format(G))  # same result
```

Best Practices:

- Always `strip()` input data
- Use `lower()` before counting DNA bases
- Prefer f-strings for formatting
- Use double quotes when text has apostrophes

Fact: f-strings — Formatted String Literals (Python 3.6+) **f-strings** are the **modern, fastest, and most readable** way to insert variables and expressions directly into strings. Just write a normal string but:

- Prefix it with **f** or **F**
- Put any Python expression inside **curly braces**

```
name = "Ram"
age = 21
G = 6.6743e-11
dna = "gcatgact"

print(f"Hello {name}, you are {age}
years old")
# Hello Ram, you are 21 years old

print(f"G = {G:.2e} N-sq-m/sq-kg")
# G = 6.67e-11 N-sq-m/sq-kg

print(f"DNA length = {len(dna)}, A
count = {dna.count('a')}")
# DNA length = 9, A count = 4

print(f"Square of 5 is {5**2}")
#  Square of 5 is 25
```

Rule: f"...variable..." or f"...expression..." That's it — use only f-strings in this lab!

Fact: Formatting Rules in Python
Use only f-strings (best and fastest method)

```
x = 1234.56789
n = 42
s = "Physics"

print(f"Value = {x}")           # Value = 1234.56789
print(f"Fixed  = {x:.2f}")      # Fixed  = 1234.57
print(f"Sci    = {x:.2e}")      # Sci    = 1.23e+03
print(f"Width  = {n:8d}")       # Width  =       42
print(f"Center = {s:^10}")      # Center =  Physics
print(f"Left   = {s:<10}!!")    # Left   = Physics  !!
print(f"Right  = {s:>10}")      # Right  =   Physics
print(f"Percent= {0.333:.1%}") # Percent= 33.3%
```

Common format codes inside {} :

- `:.nf`: n decimal places (float)
- `:.ne`: scientific, n digits
- `:width`: minimum field width
- `:>` : right-align, `:^`: center, `:<` : left-align
- `:0width`: pad with zeros
- `:%`: percentage

Physics lab favorites:

```
print(f"F = {F:.2e} N")          # scientific
print(f"m = {mass:.3f} kg")      # 3 decimals
```

Keep this page bookmarked — you'll use it in every lab!

Fact 9: Lists – Mutable

```
L = [1, 3, 2]
L.append(4)
L.reverse()
L.sort()
L
```

Fact 10: Dictionaries

```
d = {"Ram":21, "Riya":23}
d["Ravi"] = 22
d.get("Ram", "Not found")
```

Fact: List Operations & List Comprehension
Lists are **mutable**, ordered collections. You can change, add, or remove items after creation.

```
# Create a list
energies = [1.2, 5.8, 3.1, 9.7]      # physics: energy levels
bases = ['A', 'T', 'G', 'C', 'A']

# 1. Access by index (0-based)
print(bases[0])       # 'A'
print(bases[-1])      # 'A' (last item)
print(energies[1:3])  # [5.8, 3.1] (slicing)

# 2. Modify
energies[0] = 2.4
bases[4] = 'G'

# 3. Add items
energies.append(7.3)           # add to end
energies.insert(2, 4.5)        # insert at index 2
bases.extend(['T', 'C'])       # add multiple

# 4. Remove items
energies.pop()                 # remove last  7.3
energies.pop(1)                # remove at index 1
del bases[0]                   # delete by index
bases.remove('T')              # remove first 'T'

# 5. Useful methods
print(len(energies))           # length
print(sum(energies))           # total
print(max(bases))              # 'T' (alphabetically)
print(sorted(energies))        # sorted copy
energies.reverse()             # reverse in place
```

List Comprehension — Python's Superpower!
Create new lists in **one elegant line**.

```
# Traditional way (4 lines)
squares = []
for i in range(1, 6):
    squares.append(i**2)

# List comprehension (1 line!)
squares = [i**2 for i in range(1, 6)]
print(squares)        # [1, 4, 9, 16, 25]

# With condition
evens = [x for x in range(10) if x % 2 == 0]
print(evens)          # [0, 2, 4, 6, 8]
```

```python
# Physics: convert masses to weights (F = m*g)
masses = [5.972e24, 7.34e22, 1.989e30]   # Earth, Moon, Sun
weights = [m * 9.8 for m in masses]

# DNA: extract only purines (A, G)
sequence = "gCaTgAcTaTtCcTtGaC"
purines = [base.upper() for base in sequence if base.upper() in 'AG']
print(purines)        # ['G', 'C', 'A', 'T', 'G', 'A', 'C', 'T', 'A', 'T', 'T', 'C', 'C', 'T', 'T', 'G', 'A', 'C']

# Nested comprehension (advanced)
pairs = [(i,j) for i in range(3) for j in range(3)]
print(pairs)          # (0,0), (0,1), ..., (2,2)
```

Best Practices for Lab:

- Use `append()` in loops, `extend()` for multiple items
- Use list comprehension for clean, fast code
- Never modify a list while looping over it directly — use comprehension instead
- `copy()` a list before sorting if you need original: `backup = energies.copy()`

One-liner DNA counter using comprehension:

```python
dna = "gcatgactattccttgac"
count_a = sum(1 for base in dna if base == 'a')
print(count_a)    # 4
```

Control Flow & Loops

Fact 11: Branching with if-elif-else — Decision Making Python uses **if**, **elif**, and **else** to make decisions. The code inside the block runs only if the condition is **True**.

```
# Simple if
x = 10
if x > 0:
    print("Positive")

# if-elif-else
grade = 85
if grade >= 90:
    print("A")
elif grade >= 80:
    print("B")
elif grade >= 70:
    print("C")
else:
    print("Try harder")
# Output: B
```

Logic inside loops — Your favorite red/blue square!

```
from turtle import *
import time
time.sleep(1)        # pause at beginning

reset()
test = 4
while test > 0:
    if test % 2 == 0:           # even red
        color('red')
    else:                       # odd blue
        color('blue')
    forward(50)
    right(90)
    test = test - 1
done()
```

Comparison operators you must know:

- `==` → equal to
- `!=` → not equal
- `>` , `<` , `>=` , `<=`
- `in` → membership (e.g., `'a' in dna`)
- `and` , `or` , `not`

Physics example — classify particle

```
mass = 9.1e-31
if mass < 1e-27:
    print("Lepton (e.g., electron)")
elif mass < 1e-25:
    print("Meson")
else:
    print("Baryon (e.g., proton)")
```

DNA base check

```
base = 'X'
if base in 'ATGC':
    print("Valid DNA base")
else:
    print("Invalid base!")
```

Nested if (allowed, but avoid deep nesting)

```
score = 95
attendance = 80
if score >= 90:
    if attendance >= 75:
        print("Scholarship granted!")
    else:
        print("High score but low attendance")
```

Best Practices:

- Indent with **4 spaces** (never tabs)
- Use `elif` instead of multiple `else: if`
- Always end with `:`
- Test both True and False paths
- Combine with loops for powerful patterns

Fact 12: while and for Loops — Repetition with Style
Loops let you repeat code. Python has two main types: **while** and **for**.

1. while loop — repeats as long as condition is **True**
Example: Free fall with air resistance — until velocity stabilizes

```
v = 0; t = 0; dt = 0.1
g = 9.8; m = 80; c = 15      # drag coefficient
v_terminal = (m*g/c)**0.5

while abs(v - v_terminal) > 0.01:     # until nearly terminal
    F_drag = c * v**2
    a = g - F_drag/m
    v += a * dt
    t += dt
    print(f"t={t:5.1f}s and v={v:6.2f} m/s")

print(f"Terminal velocity reached = {v:.2f} m/s at t={t:.1f}s")
```

2. for loop — perfect when you know the number of repetitions

```
# Draw a square - the classic example
from turtle import *
reset()

for i in range(4):
    forward(100)
    right(90)

done()
```

3. Combine with logic — Your famous red/blue square!

```
from turtle import *
import time
time.sleep(1)        # pause at beginning

reset()
test = 4
while test > 0:
    if test % 2 == 0:           # even  red
        color('red')
    else:                       # odd  blue
        color('blue')
    forward(50)
    right(90)
    test = test - 1
done()
```

4. for loop with meaningful variable

```
# Better style
for i in range(1, 6):
    print(f"Multiplication: 5 x {i} = {5*i}")
```

5. Looping over a list (DNA example)

```
dna = "gcatgact"
for base in dna:
    print(base.upper())
```

6. range() explained

- `range(5)` → 0,1,2,3,4
- `range(1,6)` → 1,2,3,4,5
- `range(0,10,2)` → 0,2,4,6,8 (step=2)

7. Infinite loop? Use break

```
while True:
    name = input("Enter name (or 'quit'): ")
    if name == "quit":
        break
    print(f"Hello {name}!")
```

8. Skip with continue

```
for i in range(10):
    if i % 2 == 0:
        continue    # skip even numbers
    print(i)        # prints only odd
```

In the first line of the above code, we create a Python list. The rest of the code is an example of a python loop / iterative sequence, in which we make x take on the values 1,2,3 and 4 in succession. These values just mark the current iteration cycle. Note the colon, which marks the beginning of a scope and the indented lines which indicate a compound statement contained within the scope.

Best Practices for Lab:

- Use `for` when count is known
- Use `while` when condition-based
- Always update loop variable in `while`
- Indent exactly 4 spaces
- Use `range()` for clean counting

Pro tip: Your red/blue square uses **while + if** — the most powerful combo!

Turtle Graphics — Step by Step

Fact 14: Start Turtle — LOGO Style!

```
from turtle import *
reset()        # clears the canvas
```

Fact 15: Draw a Square — Manual Way

```
forward(50); right(90)
forward(50); right(90)
forward(50); right(90)
forward(50); right(90)
```

Explanation: Each line moves forward 50 pixels and turns right 90° — forms a square.

Fact 16: Draw a Square — Using for Loop

```
for i in range(4):
    forward(100)
    right(90)
done()         # keeps window open
```

Fact 17: while Loop + Logic + Color Change

```
test = 4
while test > 0:
    if test % 2 == 0:
        color('red')
    else:
        color('blue')
    forward(50)
    right(90)
    test -= 1
done()
```

Explanation: Alternates red/blue using `test % 2 == 0`

Fact 20 User-Defined Functions — Create Your Own Commands A **function** is a reusable block of code that performs a specific task. You define it once and call it anytime!

```
# Basic function
def greet():
    print("Hello St. Xavier's!")

greet()    # call it
```

Function with parameters and default values:

```
from turtle import *

def sqr(length=50, color_name="red"):
    '''Draw a square with given side and color'''
    color(color_name)
    for i in range(4):
        forward(length)
        right(90)

# Call with defaults
sqr()                    # 50px, red

# Call with custom values
sqr(100, "blue")         # big blue square
sqr(color_name="green")  # 50px green
```

Add pause using time module:

```
import time
time.sleep(1)    # pause 1 second at start of drawing
```

Save function in file (say csqr.py):

```
# File: csqr.py
from turtle import *
import time

def sqr(length=50, color_name="red"):
    color(color_name)
    for i in range(4):
        forward(length)
        right(90)
    time.sleep(1)
```

Run from shell:

```
>>> exec(open("csqr.py").read())   # old: execfile("csqr.py")
>>> reset()
>>> sqr(80, "purple")
>>> sqr(120)
```

Get help on your own function!

```
>>> help(sqr)
Help on function sqr in module __main__:

sqr(length=50, color_name="red")
    Draw a square with given side and color
```

Advanced: Return a value

```
def gravity_force(m1, m2, r):
    G = 6.6743e-11
    return G * m1 * m2 / r**2

F = gravity_force(5.972e24, 7.34e22, 3.84e8)
print(f"Earth-Moon force = {F:.2e} N")
```

Best Practices:

- Use `def` to create functions
- Add `'''docstring'''` for help
- Use default values for optional parameters
- Save reusable code in `.py` files
- Use `exec(open("file.py").read())` to load
- Always test with `help(your_function)`

Fact 21: Help is Your Friend!

```
>>> help(sqr)
>>> help(forward)
```

Exercise 1 – First 6 Problems to be submitted mandatorily by 27th November, 2026 on MS Teams

1. Invoke Python interpreter. Read two complex numbers and print their ratio and product of first with conjugate of second.

2. Programming Exercise: Star Triangles using String Alignment Methods
Write a Python program that reads a positive integer n from the user and then prints **three star (*) triangles** of height n, each built **exclusively** using the string methods `.ljust()`, `.rjust()`, and `.center()` (**no** manual spacing like `' ' * k` allowed).
The output must look **exactly** like the examples below when $n = 5$:

1. Left-justified triangle (using `.ljust(n)`)

```
*
**
***
****
*****
```

2. Right-justified triangle (using `.rjust(n)`)

```
    *
   **
  ***
 ****
*****
```

3. Center-justified (symmetric) triangle (using `.center(2*n-1)`)

```
    *
   ***
  *****
 *******
*********
```

Requirements & Hints

- Read n with `n = int(input("Enter height: "))`
- For left-justified: `print(('*' * i).ljust(n))`
- For right-justified: `print(('*' * i).rjust(n))`
- For centered: use width $2n - 1$ and `2*i-1` stars per row i
- Print the labels and blank lines exactly as shown above

3. Print sequence 0,1,2,...,25 using modulo operator.

4. Load `physconsts.py` and read help on its contents.

5. Draw a triangle on turtle canvas using a for loop.

6. 20-Step Random Walk using random module
Use the turtle to simulate a **20-step random walk** on the screen. At each step, the turtle must randomly choose one of four directions: **0° (East), 90° (North), 180° (West), or 270° (South)** and move forward by a fixed length (e.g., 30 units).
Hint: First import the random module: `import random`
Then inside the loop, use: `angle = random.choice([0, 90, 180, 270])` `right(angle)` or `setheading(angle)` before moving `forward(30)`
Write the complete code using a `for` loop with `range(20)`. Make each walk colourful by randomly changing pen color too!
Sample start:

```
from turtle import *
from random import choice
reset()
colormode(255)
speed(0)

for i in range(20):
    color(choice(['red','blue','green','purple']))
    setheading(choice([0,90,180,270]))
    forward(30)

done()
```

Now modify it to make your own beautiful random dance!

7. Define a function to draw any regular polygon. Test for several polygons.

8. Generate multiplication table of any number (1–100).

9. Print butterfly wings pattern.

10. Draw a Beautiful 5-Pointed Star — Fill, Run, Play!
Complete the code below by filling in the blank spaces to draw a golden 5-pointed star using only one loop.

```
from _________________ import *
reset()
speed(_____)
color("________")
pensize(____)

for i in range(_____):
    forward(_____)
    ________(144)

done()
```

Your task:

1. Fill in the blanks to make a perfect star
2. Run it — admire the magic!
3. Now experiment:
   - Change 144° to 72° → what shape?
   - Try 120° → what happens?
   - Use range(7) and 360/7 = 51.4° → magic again?
   - Add random colors like in the random walk!

Exercise 2 – Advanced (8 Problems)

11. Count a, c, g, t in DNA string `"gcatgactattccttgac"`

12. Print reverse of the DNA string.

13. Print 6th base from the end.

14. Find all positions of 'a' in the DNA.

15. Write function for reverse complement of DNA.

Reverse Complement of a DNA Sequence Using `str.maketrans` and `translate()`

Consider the DNA sequence

`ATGCGTTAAC`.

To compute its *reverse complement*, we follow these steps:

1. **Reverse the sequence:** write the bases in the opposite order (from 3' to 5').
2. **Complement each base:** use the base-pairing rules

$$A \leftrightarrow T, \qquad C \leftrightarrow G.$$

In Python, we can do this efficiently using the built-in functions `str.maketrans` and `translate`:

- `str.maketrans("ACGT", "TGCA")` creates a *translation table* that maps each character in the first string to the corresponding character in the second:

$$A \mapsto T,\ C \mapsto G,\ G \mapsto C,\ T \mapsto A.$$

- The method `dna.translate(table)` then applies this mapping to every character in the string `dna`, producing the complemented DNA sequence.

Python Program

```
# DNA sequence (new example)
dna = " ATGCGTTAAC"

# Step 1: reverse the sequence (3' -> 5')
rev = dna[::-1]

# Step 2: create translation table for complements
# A <-> T, C <-> G
comp_table = str.maketrans("ACGT", "TGCA")

# Step 3: complement the reversed sequence
rev_comp = rev.translate(comp_table)

print("Original sequence        :", dna)
print("Reversed sequence        :", rev)
print("Reverse complement       :", rev_comp)
```

For this example, the program prints the reverse complement

`GTT AACGCAT`,

i.e. `GTT AACGCAT` (without spaces), which is the 5'→3' sequence of the strand complementary to the original.

16. Replace all 'a' with 'A' and revert back.

17. Join two DNA strings with `join()`.

18. Calculate ratio of gravitational to electrostatic force between two protons at Bohr radius, by importing the `phyconsts.py` file to your workspace. Also, try with `exec(open("phyconsts.py").read())`.

References

[1] W3Schools, *Python Tutorial*, Available at: `https://www.w3schools.com/python/`. Accessed 21 Feb 2025.

[2] Abhijit Kar Gupta, *Python Programming*, 2nd Edition, University Press, Hyderabad, 2020.

[3] Python Software Foundation, *Python Documentation*, Available at: `https://docs.python.org/3/`. Accessed 21 Feb 2025.

[4] Al Sweigart, *Automate the Boring Stuff with Python*, 2nd Edition, No Starch Press, San Francisco, 2020.